Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

SICKSense Final Demo

Team Number: 33
Team members:
Jason Agnew, Josiah Faircloth
Samuel Ramos, Paavan Bagla
TA: Vishwam Raval
Sponsor: SICK

Project Summary

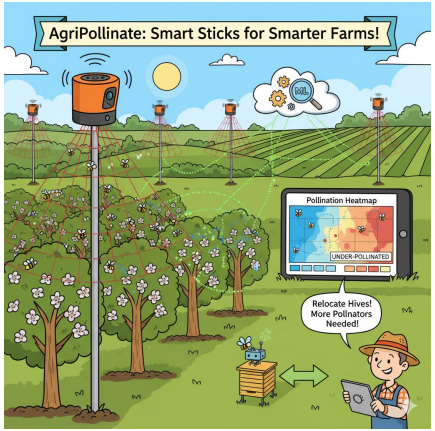
Problem statement:

- Farmers lack reliable, real-time insights into pollination activity. Existing methods are reactive, labor-intensive, and often inaccurate, leading to missed pollination deficits that directly reduce crop yield and quality.

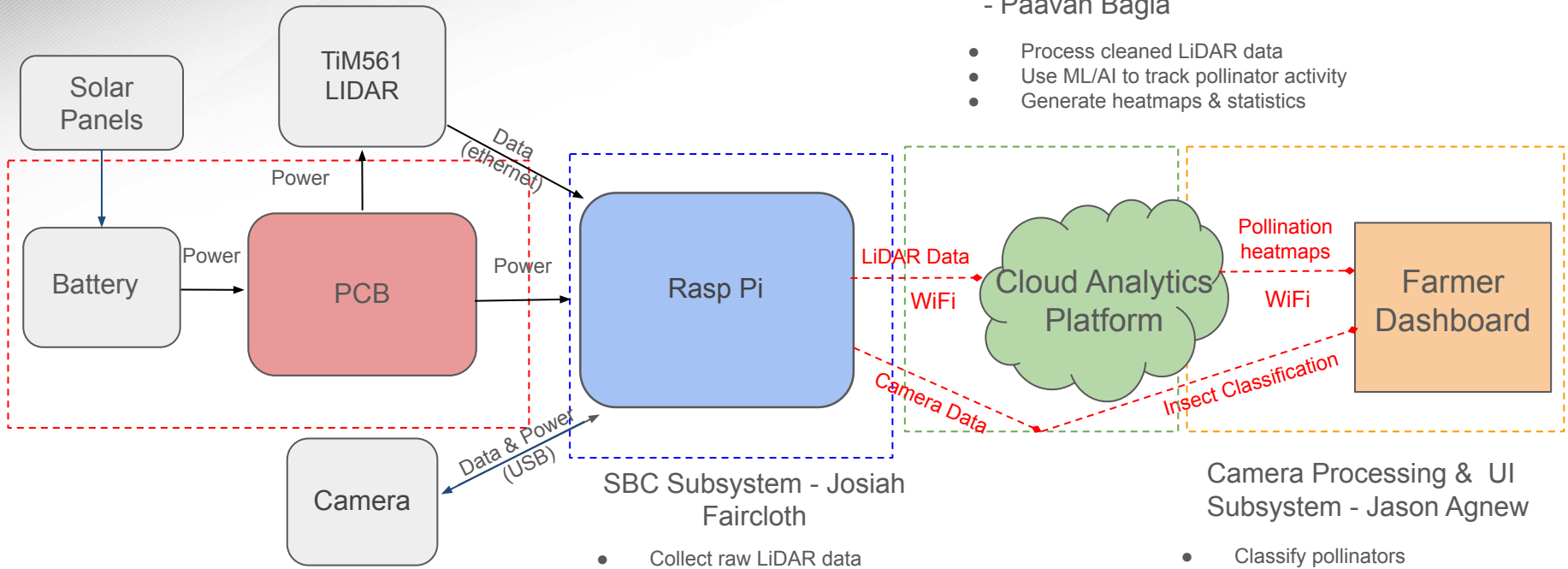
AgriPollinate

Our Solution:

- AgriPollinate is a Light Detection and Ranging (LiDAR)-powered monitoring system that continuously tracks pollinator activity and generates easy-to-read heatmaps.
- Farmers can access these heatmaps through a simple dashboard, allowing them to quickly identify under-pollinated areas, optimize hive placement, and improve crop yield and quality.



System Overview



LiDAR Processing Subsystem - Paavan Bagla

- Process cleaned LiDAR data
- Use ML/AI to track pollinator activity
- Generate heatmaps & statistics

Power Subsystem - Samuel Ramos

- Supply power to physical components
- Charge Controller for the Solar Panel

SBC Subsystem - Josiah Faircloth

- Collect raw LiDAR data
- Detect objects in LiDAR data
- Capture images
- Transmit cleaned data & images

Camera Processing & UI Subsystem - Jason Agnew

- Classify pollinators
- Deliver heatmaps & statistics to easily digestible user interface

Single Board Computer (SBC) Subsystem

Josiah Faircloth

Designed:

- Image capture & transmission program
  - Captures images via camera attached to Raspberry Pi and transmits using TCP
- LiDAR data collection program
  - Connects to TiM561 using TCP, receives scan data, and writes to a file
- LiDAR data object detection algorithm
  - Reads scan data file, groups data points into objects, and triggers when a new object is detected
- Integrated program unifying all previous modules including a watchdog timer.

Tested:

- TCP/SSH connections with raspberry pi
- Image capture & transmission latency
- TiM561 connection & data parsing
- Object detection algorithm accuracy

SBC Subsystem

Josiah Faircloth

Validation:

- >1 hr stable connection between pi and device on same network
- First Image ~1ms after trigger, average latency between pictures is ≤ 200ms
- >1 hr successful connection & data parsing with LiDAR
- Roughly 8/10 successful detection with simulated insect
- Roughly 3/10 detection triggers are false positives

Prepare SBC for remote software development	Connect to the SBC using an SSH client	>1 hr stable connection	Raspberry Pi 5, development laptop, local WiFi/Ethernet network,	Achieved	Josiah
Validate reaction latency	Python script recording start & capture times	avg latency ≤ 200ms	Raspberry Pi 5, USB camera, Python environment	Achieved	Josiah
Confirm stable connection to LiDAR	Python script to connect to LiDAR and parse incoming data.	run for 1 hr w/o connection loss or parsing errors	Raspberry Pi 5, TiM561 LiDAR, Ethernet connection	Achieved	Josiah
Validate object detection	simulated pollinator in LiDAR's field of view in controlled environment	< 30% false positives / negatives	Raspberry Pi 5, TiM561 LiDAR, simulated pollinator	Achieved	Josiah

SBC Subsystem

Josiah Faircloth

404 Tasks:

- Integrate TCP server/client with LiDAR & camera processing subsystems
- Optimize algorithm for performance and memory use to reduce strain on Raspberry Pi
- Continued testing and optimization of object detection algorithm
- Implement memory management system for pi
- Field testing over extended periods with real bees

LiDAR Data Processing & Pollination Heatmaps System:

Subsystem Summary — Paavan Bagla

Goal:

- Detect, identify, and visualize bee activity using LiDAR data to generate pollination heatmaps that inform autonomous robot behavior.

Design & Implementation:

- Integrated RViz + Gazebo simulation environment for sensor and visualization testing
- Built ROS2 pipeline for LiDAR /scan ingestion, bee point extraction, clustering, and heatmap generation
- Implemented both real-time and cumulative heatmap logic for visualizing bee activity

LiDAR Data Processing & Pollination Heatmaps System:

System Design (Algorithms Used) — Paavan Bagla

- **Polar-to-Cartesian Conversion & Filtering:** Transforms LiDAR (angle, range) pairs into 2D (x, y) coordinates and removes invalid or noisy readings.

- **DBSCAN clustering:** Uses density-based clustering to group nearby LiDAR points, helping distinguish actual bee detections from random noise.

- **Gaussian smoothing:** Applies Gaussian filtering using SciPy (or fallback) to smooth the heatmap grid and highlight consistent regions of bee activity.

- **Heatmap accumulation logic:** Updates two maps:
  - **Real-time heatmap** → live detections per frame
  - **Cumulative heatmap** → tracks total **pollinating bee activity** and **count** over time.

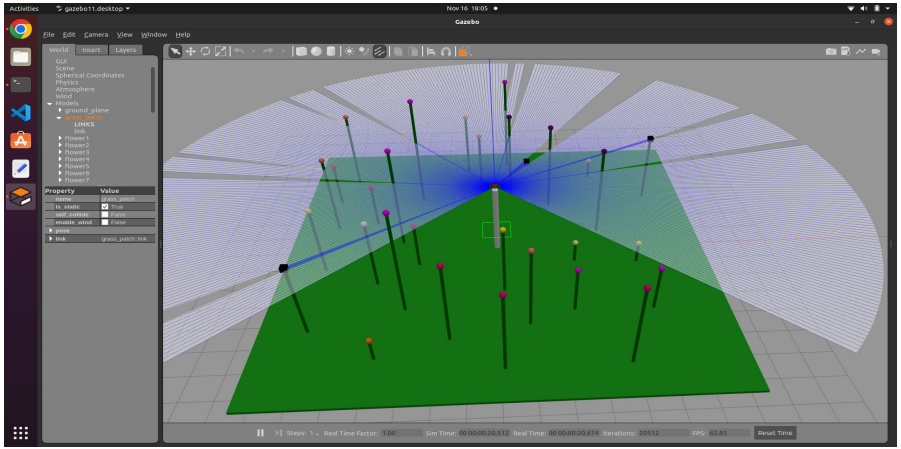
- **ROS2 integration for real-time data streaming:** Subscribes to /scan, processes detections, and publishes heatmaps to RViz.

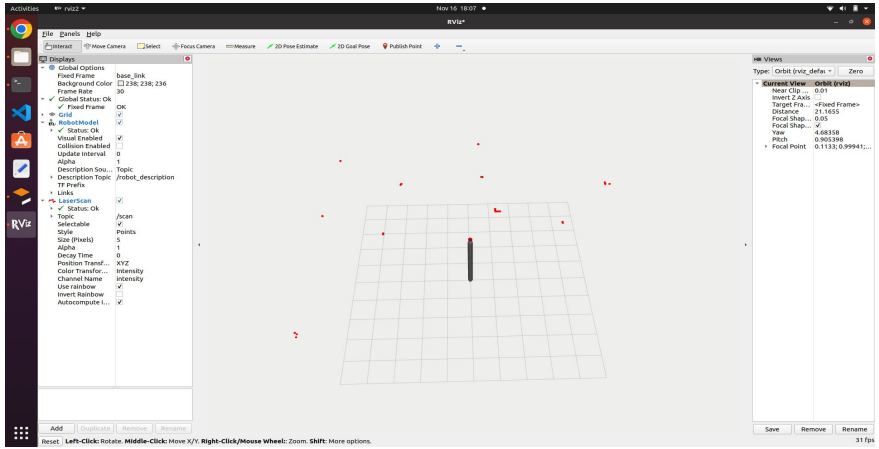
Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

LiDAR Data Processing & Pollination Heatmaps System: Simulated LiDAR Detection in Gazebo & RViz

Paavan Bagla

Validated ingestion of simulated LiDAR scan data & Successfully visualized real-time bee detections in RViz





LiDAR Data Processing & Pollination Heatmaps System:

Validation Plan — Paavan Bagla

Validtion Plan

Requirement / Objective	Validation Method	Metric / Target	Test Environment / Tool	Status	Owner
Validate LiDAR processing pipeline	Convert /scan to (x, y), filter invalid data, cluster using DBSCAN	Correct cluster detection with <5% false positives per frame	ROS2 node test on simulated LaserScan	Achieved	Paavan
Verify real-time heatmap correctness	Compare real-time heatmap hotspots with known moving bee positions in Gazebo	≥95% visual match between bee paths and heatmap	Python + Gazebo simulation	Achieved	Paavan
Validate system performance	Measure update speed (scan → clustering → heatmap)	Real-time performance with updates ≤ 1 second	ROS2 real-time test with simulated LiDAR	Achieved	Paavan
Validate cumulative heatmap accumulation	Track persistent cluster detections and cumulative activity	Proper accumulation + correct pollinating-bee count	Python visualization module	Achieved	Paavan

Testing & Validation:

- Verified LiDAR scan processing and visualization in RViz
- Tested with simulated bees in custom Gazebo environment
- Heatmaps reliably highlight real-time and cumulative pollination activity
- Counting + clustering algorithms run consistently in real time

404 Tasks: LiDAR & Heatmap Subsystem

Paavan Bagla

- **Machine Learning Integration**
  - Train a model to classify LiDAR points as bees vs. non-bees
  - Detect if a bee is pollinating
  - Integrate ML predictions into the existing real-time pipeline
- **Enhanced Heatmap Visualization**
  - Enhance real-time and cumulative heatmap visuals for better user experience
- **Testing & Validation**
  - Test ML-enabled subsystem with larger simulated swarms
  - Validate real-time performance with multiple moving bees
  - Confirm correct pollinator counts and heatmap hotspot accuracy
- **System Integration**
  - Use actual SICK TiM561 2D LiDAR data instead of simulated scans
  - Integrate with other subsystems and finally display heatmaps and pollination analytics on the UI dashboard

Camera Processing & UI Subsystem

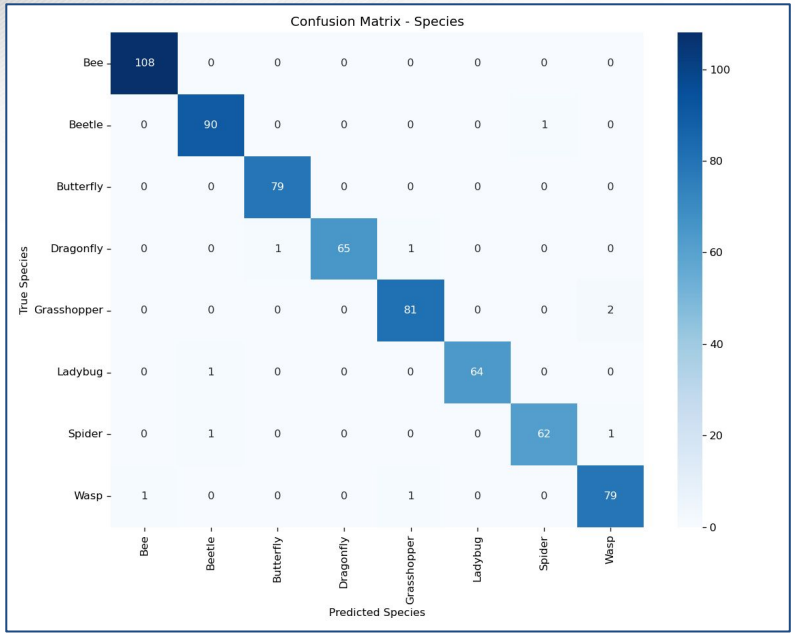
Jason Agnew

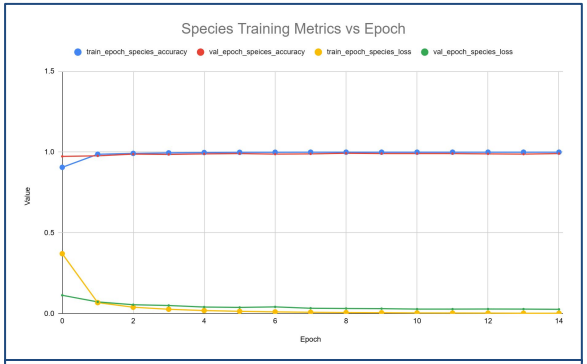
Real-time Data Update Functionality	Simulate update testing; dashboard monitoring	Dashboard reflects live data after updat	Dashboard with simulated update button	Achieved	Jason
Web-based Image Classification Capability	Image prediction testing in browser	Images classified using pre-trained model	Web page with TensorFlow.js; sample images	Achieved	Jason
Initial Model Training	Model evaluation using test set; dashboard check	Accuracy ≥60%; dashboard shows test classifications	Model training environment; dashboard	Achieved	Jason
Intermediate Model Validation	Validation on simulated inputs; heatmap rendering	Accuracy ≥70%	Model validation environment; simulated inputs	Achieved	Jason
Dashboard Responsiveness	Dashboard refresh test; alert simulation	Refresh ≤10s; simulated pest/pollinator alerts correct	Dashboard; alert simulation tool	Achieved	Jason
Robustness Test	Sequential input testing	Accuracy ≥80%; no failures under sequential inputs	System with simulated sequential inputs	Achieved	Jason
Final Validation and Documentation	Final model test; demo/documentation review	Accuracy ≥85%; documentation/demo completed	Final model; documentation/demo environment	Achieved	Jason

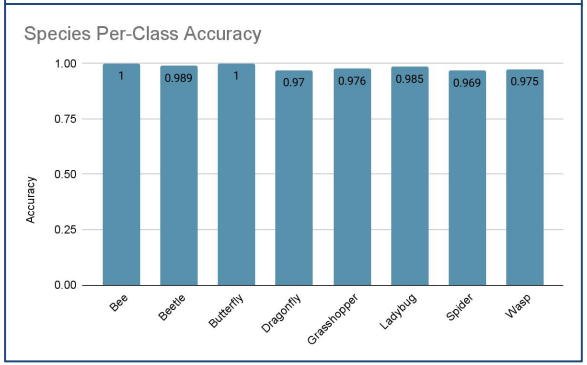
Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

Species

Number of species classes: 8
Input image size: 224x224x3
Test images evaluated: 638
Model parameter count: 2,269,513 (Trainable: 11,529)





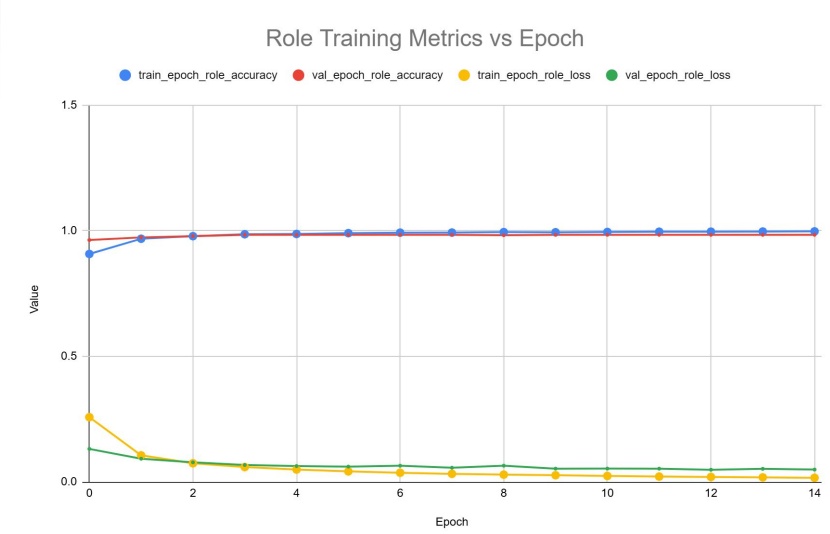


<u>Test Performance:</u>

Species Test Accuracy: 98.43%
Species Macro Precision: 0.9850
Species Macro Recall: 0.9830
Species Macro F1 Score: 0.9839

Roles

Pollinator/Pest confusion matrix:
[[316 3]
[8 311]]



<u>Test Performance:</u>
Role Test Accuracy: 98.28%
Role Precision: 0.9904
Role Recall: 0.9749
Role F1 Score: 0.9826

Solar Power Management Circuit

Samuel Ramos

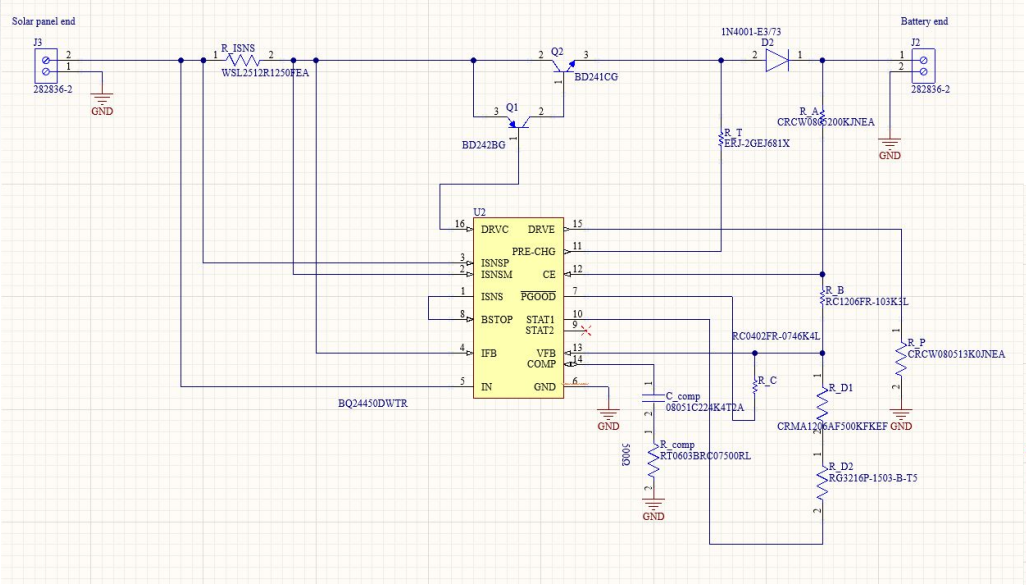
403 Accomplishments Summary:

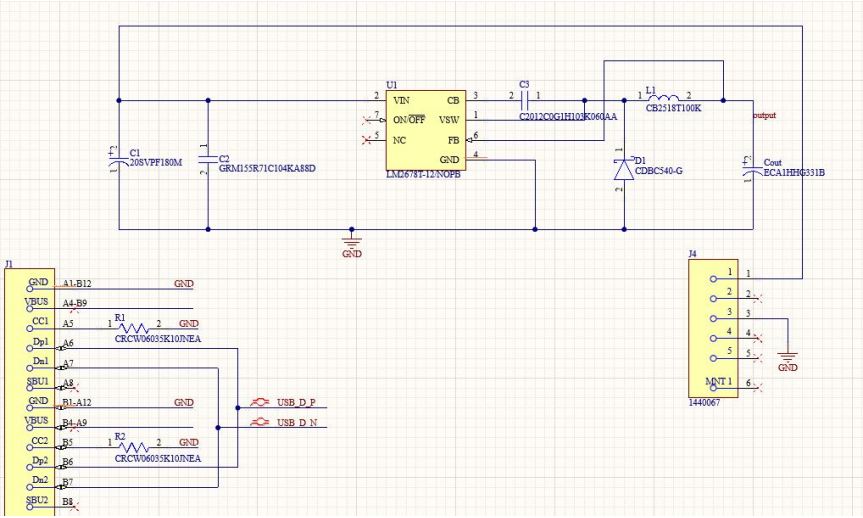
- Schematics for Step-down converter & Charge Controller has been design, passed the validation check and have been completed
- Learned a new software, Altium, in order to complete the schematics and more importantly the design of the PCB, which combines the two schematics
- PCB design has arrived and been validated to match the design in Altium

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

Solar Power Management Circuit

Samuel Ramos

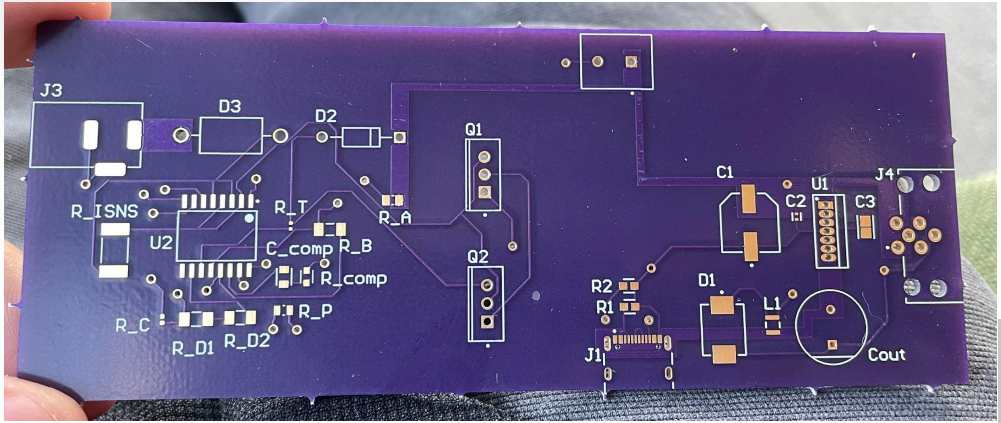


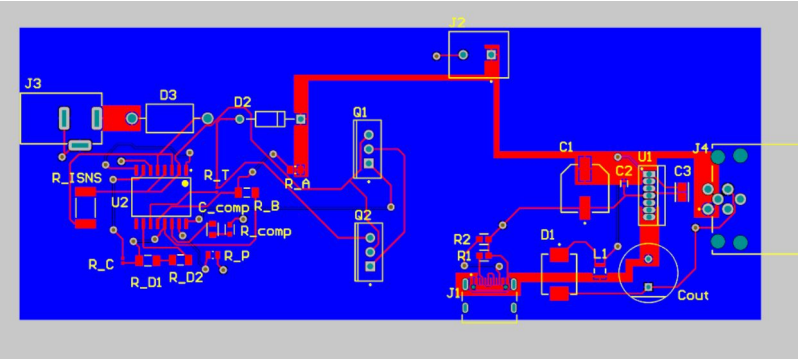


Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Solar Power Management Circuit

Samuel Ramos





Solar Power Management Circuit

Samuel Ramos

Remaining Tasks:

- Soldering of parts onto PCB
- Testing of completed PCB (Power, I/O)
- Validation of testing compared to plan

Key Design activity to be completed in 404:

- Possible Redesign of Schematics and therefore PCB based on results of testing
- Enclose for PCB and components

Remaining Tasks: Camera Processing & UI Subsystem
Jason Agnew

Remaining Tasks Before Demo:

- Complete bounding-box annotations for the training dataset to finalize the labeled image set.
- Begin training the prebuilt MobileNet model using the fully annotated dataset.
- Evaluate model performance first using images from the dataset, then testing with images captured from the actual camera hardware.
- Test the end-to-end subsystem with multiple image bursts to validate dashboard refresh rate and real-time graph updates.

404 Tasks:

- Scale the model to support multi-insect detection and classification within a single image.
- Explore methods for integrating image-based classification results with LiDAR object-tracking to improve overall pollinator identification and reduce false counts.

Thank you everyone for your attention!
Feel free to ask any questions!

Team 33 - AgriPollinate

Team Members:

Jason Agnew

Josiah Faircloth

Samuel Ramos

Paavan Bagla

TA: Vishwam Raval

Sponsor: SICK